StatGuard: Reproducible Statistical Analysis by Separating LLM Orchestration from Verified Computation

Cheng Peng

Independent Researcher

Abstract

Large language models (LLMs) are increasingly used as conversational data analysts: a user describes a dataset and a question in natural language, and the model selects a statistical method, runs it, and reports a result. This convenience comes with a structural cost. Because LLMs generate output by sampling, the *same* dataset and the *same* question can yield *different* statistical methods and *different* numerical conclusions across independent runs, depending on whether the model happens to check an assumption on a given attempt. In settings where a reported p-value or effect size must be defensible and reproducible—scientific research, clinical analysis, and regulated decision-making—this variability is a serious liability.

We present **StatGuard**, a framework that preserves the natural-language convenience of an LLM analyst while removing statistical computation from the LLM entirely. StatGuard enforces a separation of responsibilities: the LLM acts only as an *orchestrator* that interprets the request and selects which analysis to run, while every statistic is produced by a fixed library of hardcoded methods that are cross-validated, case by case, against established reference implementations (`scipy`, `statsmodels`). The LLM cannot compute or alter a numerical result. We describe the architecture, including assumption-aware method routing and an anti-fabrication "claims ledger" that prevents the model from reporting an uncomputed statistic. We validate the deterministic engine against a 246-case benchmark with known ground truth (246/246 agreement), and we present six case studies contrasting StatGuard with a widely used commercial LLM analyzer. In one paired-data example, repeated runs of the commercial system produced primary p-values differing by roughly 500-fold, solely according to whether that run happened to test an assumption, whereas StatGuard returned an identical, assumption-appropriate result on every run.

1 Introduction

The capabilities of large language models have made a compelling interface possible: a researcher uploads a dataset, asks a question in plain language—*"did scores change between the two time points?"*—and receives, within seconds, a chosen statistical test, a computed result, and a written interpretation. Several commercial products now offer this experience, and it is increasingly used not only for casual exploration but for analyses that inform real decisions.

The convenience, however, rests on an uncomfortable foundation. When an LLM both *chooses* a statistical procedure and *computes* its result, both steps inherit the model's stochasticity. The choice of test may vary from run to run; the reported numbers may be generated text rather than the output of an actual computation; and the rigor of the analysis—whether assumptions are checked, whether effect sizes and their uncertainties are reported, whether multiple-comparison issues are flagged—depends on what the model happens to produce on a given attempt. The consequence is that an LLM analysis can be *correct on Monday and subtly wrong on Tuesday*, with no change in the input and no signal to the user that anything differed.

This is not a hypothetical concern. To make it concrete, consider a paired pre/post dataset whose differences are non-normal (a few large responders), analyzed with the lay prompt *"Compare pre_score and post_score (same subjects). Did scores change?"*. We submitted this identical prompt and dataset to a leading commercial LLM analyzer in five independent sessions. In four of the five runs, the system applied a paired t-test and reported $p = 0.009$. In the fifth run, the system first tested the normality of the differences, found it violated, and switched to a Wilcoxon signed-rank test, reporting $p = 0.000018$—a primary p-value differing from the other runs by a factor of roughly 500. The fifth run's own explanation stated that the Wilcoxon test was "more appropriate ... than a paired t-test," implicitly conceding that the other four runs had used a less suitable method. The only thing that varied across runs was whether the model spontaneously checked an assumption.

We emphasize that this example is an *illustration*, not a controlled large-sample study; the systematic, large-scale documentation of LLM analytical irreproducibility has been carried out elsewhere [1, 5]. Our point in raising it is to motivate a design question: if the unreliability arises precisely because a sampling-based model is in charge of statistical decisions and computations, then no amount of model improvement removes the root cause—a more capable model still samples, and still may or may not check an assumption on a given run. The problem is structural, and a structural problem invites a structural solution.

We argue that statistical computation should be removed from the LLM altogether. The appropriate role for the model is *orchestration*—understanding a natural-language request and selecting an appropriate, pre-validated analysis—while the statistics themselves should be produced by deterministic methods whose correctness can be verified independently. This preserves what makes the LLM interface valuable (a user need not know which test to run) while restoring what makes traditional statistical software trustworthy (the same input always produces the same, correct output). The resulting system aims to be, in a phrase, *as adaptable as an LLM and as reproducible as a fixed statistical routine*—a combination offered neither by general-purpose LLM analyzers (which compute statistics stochastically) nor by traditional packages such as SPSS or base `scipy`/`statsmodels` (which are reproducible but require the user to choose the method and verify its assumptions).

We present **StatGuard**, a framework realizing this separation, and make the following contributions:

1. **An architecture that separates orchestration from computation.** An LLM supervisor selects one analysis action at a time from a registry of deterministic statistical plugins; the plugins compute and self-describe their results; the LLM never performs a calculation. (Section 3)

2. **Assumption-aware, deterministic method routing.** Rather than relying on the LLM to remember to check assumptions, method selection is governed by explicit, inspectable rules encoded in the engine—for example, variance-homogeneity and normality checks that route between parametric and rank-based procedures. (Section 3)

3. **An anti-fabrication claims ledger.** Plugins emit structured, verified claims; the LLM may only reference them by identifier, and a rendering layer substitutes the verified wording into the final report. The model is therefore unable to report a statistic it did not actually compute. (Section 3)

4. **Independent numerical validation at scale.** We validate every plugin against a 246-case benchmark with known ground truth, checking each computed quantity against an independent `scipy/statsmodels` calculation; all 246 cases agree. (Section 4)

5. **Comparative case studies.** Across six analysis scenarios—using the same prompt for both systems, ranging from lay questions that do not hint at a method to prompts that explicitly request rigor—we contrast StatGuard with a widely used commercial LLM analyzer, focusing not on single-attempt capability but on reproducibility and on whether standard rigor is applied even when not prompted. (Section 4)

The remainder of the paper reviews related work (Section 2), details the system design (Section 3), reports validation and case studies (Section 4), and discusses scope, limitations, and implications for analysis in high-stakes settings (Section 5).

2 Related Work

Reproducibility of LLM-driven data analysis. A growing body of empirical work documents that LLM-generated data analysis is difficult to reproduce. Cui and Alexander [1] show, across many independent runs of the same prompt and data, that LLM analyses diverge in both method and conclusion. Zeng et al. [5] introduce an analyst–inspector framework for evaluating the reproducibility of LLMs in data-science tasks and likewise find reproducibility to be low. These studies establish the *prevalence* of the problem through large-scale measurement. Our work is complementary: rather than further quantifying the failure, we propose and validate an architecture intended to eliminate it for the domain of statistical inference, and we use small, illustrative case studies only to motivate and demonstrate that architecture.

Constraining LLM and agent workflows for reproducibility. A parallel line of work seeks to make LLM-driven scientific and analytical workflows auditable by constraining how agent actions are executed. Such frameworks enforce deterministic, traceable, and replayable execution of the steps an agent takes [3]. This addresses an important dimension—given a recorded run, one can replay it—but replayability is distinct from *statistical correctness*. A workflow can be perfectly replayable and still have selected an inappropriate test, skipped an assumption check, or reported a number that does not match an independent computation. StatGuard targets this second dimension directly: it constrains not merely *that* a computation can be reproduced, but *what* computation is performed and *whether its output is numerically correct*, validated against reference implementations.

Traditional statistical software. Established packages such as SPSS, SAS, and the `scipy/statsmodels` ecosystem [4, 2] are deterministic and trustworthy: a given procedure on given data always returns the same result. Their limitation, in the present context, is the converse of the LLM's: they cannot interpret an open-ended natural-language request and decide *which* analysis is appropriate. The user must already know the right method and must check its assumptions. StatGuard does not replace these libraries; it builds on them, delegating computation to them (and to hardcoded, reference-checked implementations) while adding the orchestration layer that selects among them.

Positioning. StatGuard occupies the space between these lines of work. From the LLM-analyst paradigm it retains natural-language orchestration; from traditional statistical software it retains determinism and verifiability; and unlike execution-replay frameworks, its guarantees concern the statistical content of the analysis, not only its reproducibility as a sequence of actions. The design rests on a single principle: the LLM decides *what* to analyze, and a verified, deterministic engine decides *how*—and actually does the computing.

3 System Design

StatGuard is organized around a single principle: the language model decides *what* analysis to perform, and a deterministic, independently verified engine decides *how* and performs the computation. This section describes how that separation is realized.

3.1 Architecture overview

An analysis proceeds as a sequence of discrete steps coordinated by a directed graph of nodes. The model-facing decision point is a single **supervisor** node, which, given the current context, selects exactly one next action—either a call to one analysis tool or a decision to conclude. Crucially, the supervisor selects an action; it does not compute its result.

Each proposed action passes through a **verification** node before execution. Verification checks the action against the dataset profile and current state—for instance, that the referenced columns exist and that the requested tool is applicable—and assigns a status (allowed, needs-review, or rejected). Only allowed actions reach the **execution** node, where the corresponding deterministic plugin actually runs; rejected actions return a structured error to the supervisor rather than producing a result. This means that a routing mistake by the model—selecting a tool with a mistyped column name, say—is caught and surfaced, not silently computed. A separate quality gate evaluates whether the evidence required by the user's request has in fact been produced before a final answer is permitted, preventing the system from concluding prematurely.

The consequence of this structure is that the model's stochastic choices are confined to *which* tool runs and *with what arguments*; everything downstream of that choice—the computation, its verification, and the wording of the reported numbers—is deterministic.

3.2 The deterministic statistics engine

Statistical computation is implemented as a registry of 25 tool plugins covering group comparison (parametric and non-parametric, with post-hoc procedures and multiple-comparison correction), correlation, association, paired comparison, multiple linear regression with diagnostics, power analysis, and supporting description, data-preparation, and reporting tools. Each plugin computes its result with hardcoded methods built on established libraries (`scipy`, `statsmodels`) and hand-implemented formulae where appropriate (for example, Welch–Satterthwaite degrees of freedom, Cohen's d and Hedges' g with confidence intervals). No plugin invokes a language model; given the same data and arguments, a plugin always returns the same result.

Plugins are written defensively, because their arguments originate from the model's choices. Missing or non-existent columns, non-numeric data, degenerate inputs (empty groups, single rows, all-missing data, constant columns, infinities), and invalid parameters return a structured, explanatory result rather than raising an exception. This robustness is itself verified by a dedicated test suite.

3.3 Assumption-aware routing

A central design decision is that statistical assumption checks are encoded as explicit, inspectable rules in the engine, rather than entrusted to the model's discretion. The model selects, for example, "compare these groups"; the engine then determines the appropriate test. Concretely, the group-comparison plugin tests variance homogeneity (Levene's test) and routes between classical and variance-robust procedures (Welch or Alexander–Govern) accordingly; it tests normality (Shapiro–Wilk) per group and, when normality is violated in conjunction with a small sample (group $n < 30$)

or strong skew (|skewness| ≥ 1.5), switches to a rank-based non-parametric alternative. Post-hoc comparisons under unequal variance use Games–Howell rather than Tukey's HSD.

Because these rules live in the deterministic engine, the decision to check an assumption—and the action taken on its result—does not depend on whether the model "remembered" to do so on a given run. This is the mechanism by which StatGuard converts an assumption check from a behavior the model might or might not exhibit into a guarantee of the system.

3.4 Verification and the anti-fabrication claims ledger

Even when the correct computation has been performed, a language model writing the final report could in principle misstate a number. StatGuard prevents this with a **claims ledger**. Each plugin emits structured *claims*—significance, effect size, direction, test statistic, and session-level warnings—generated mechanically from its results. Every claim carries an identifier and a record of which analysis run produced it. The wording of a claim is produced solely by an authoritative rendering routine operating on the claim's structured fields; the model may *reference* a claim by its identifier in its narrative but cannot change what the claim says. The reported numbers are therefore tamper-proof with respect to the model: the model can describe and contextualize a result, but the exact statistics inserted into the report are those the engine computed.

3.5 Reproducibility and provenance

Each analysis is tied to an explicit data version, identified by the content of the analysis-ready dataset, and every computation is recorded as an analysis run linked to that version. The final report records how the analysis-ready dataset was constructed, including data-source provenance where applicable, and the framework can export a reproducibility manifest together with APA-style methods text. Combined with the determinism of the engine, this provenance means that an analysis can be re-executed to yield an identical result, and that any reported quantity can be traced to the specific run and data version that produced it.

4 Validation

We validate StatGuard along two axes. First, we establish that the deterministic engine computes *correct* statistics, by checking its output against independent reference implementations across a large benchmark (Section 4.1). Second, we use a set of case studies to illustrate the reproducibility and rigor properties that motivate the design, contrasting StatGuard with a widely used commercial LLM analyzer (Section 4.2).

All StatGuard results reported here are produced by a deterministic pipeline and can be reproduced from the project's public repository; the benchmark of Section 4.1 is regenerated from a fixed random seed, and the case studies of Section 4.2 are driven by the scripts and datasets provided there. The session records for the commercial LLM analyzer are archived as supplementary material. Because that system's output is not reproducible across runs—the very property under study—these records stand as archived one-time observations rather than as a reproducible procedure.

4.1 Numerical correctness

The engine is validated against a 246-case benchmark in which each case is a statistical scenario with known ground truth. For every case, the plugin's computed quantities are compared against an independent computation of the same quantity using `scipy/statsmodels`. The benchmark spans the engine's inferential surface—group comparisons (parametric and non-parametric), correlation, association, paired comparison, and regression—across a range of sample sizes, distributional shapes, and degrees of variance heterogeneity. All 246 cases agree with the reference implementation (Table 1). This provides case-by-case evidence that the statistics StatGuard reports are numerically correct, not merely plausible.

Table 1: Numerical validation of the deterministic engine against reference implementations.

Analysis family	Cases	Agreement
Group comparison (parametric & non-parametric)	218	218 / 218
Regression and diagnostics	9	9 / 9
Paired comparison	8	8 / 8
Correlation	7	7 / 7
Association (chi-square)	4	4 / 4
Total	**246**	**246 / 246**

4.2 Case studies: reproducibility and rigor

We constructed six analysis scenarios, each embedding a standard methodological consideration—unequal variances, an influential observation, multiple comparisons, non-normality, a paired design, or effect-size reporting—that a careful analyst would address. For each scenario we used the *same* prompt for both systems. The prompts vary in how much they signal: some are lay questions that name neither a method nor a caveat (for example, *"Compare `response_time` between the placebo and drug arms and tell me whether the difference is statistically significant"*), while others explicitly ask for rigor or completeness (for example, *"...report everything I would need for a rigorous write-up"* or *"...tell me how big the difference is, not just whether it is significant"*). We submitted each scenario to StatGuard and to a widely used commercial LLM analyzer.[1]

On several scenarios—particularly those whose prompts explicitly requested rigor—both systems performed competently, and we report these as ties. Capable models, especially when prompted to be careful and run only once, often handle an isolated scenario well. The properties that distinguish the systems, and that motivate StatGuard's design, are two: whether standard rigor is applied *when the prompt does not ask for it*, and whether the result is *reproducible across runs*. These surface in the two scenarios examined below.

Table 2 summarizes the six scenarios and, for each, whether we judge the two systems' single-run outputs to be comparable (a tie) or to differ.

The systematic gap (Scenario 4). Here the prompt was a lay question—whether response time differed between two arms—with no request to check assumptions. Across five independent runs, the commercial analyzer applied a Welch t-test in all five ($p = 0.147$) and checked normality in none, despite one arm being strongly right-skewed. This is not run-to-run variation but a consistent omission: the same appropriate check was absent every time. StatGuard tested normality per

[1] We refer to the comparison system generically rather than by product name. Our claims concern the *paradigm* of using a language model as the statistician—selecting and computing the analysis—rather than any specific product, and product behavior changes across versions and over time. The archived session records provided as supplementary material identify the system and the date of testing.

Table 2: Six case studies. The same prompt was used for both systems in each scenario.

#	Scenario	StatGuard	Commercial LLM analyzer	Outcome
1	Three groups, unequal variance (rigorous write-up requested)	Levene check → Alexander–Govern ANOVA (p=0.0046), Games–Howell post-hoc, η^2, variance ratio reported	Shapiro + Levene checked → Welch ANOVA (p=0.0047) + Holm post-hoc, Hedges g; same conclusion	Tie
2	Regression, influential point (asked what to watch for)	OLS + influence diagnostics + Breusch–Pagan + HC3 robust SE computed	Reported fit; flagged non-normal residuals, heavy tails, possible influential points; recommended robust checks	Tie
3	Five tests, all noise	Negligible effect sizes (Hedges $g\approx-0.19$); correctly found no effect	Five Welch t-tests; correctly found none significant; did not raise multiple-comparison correction	Minor edge: StatGuard
4	Two groups, non-normal (lay prompt)	Shapiro (violated, p=0.002) → rank-based context; Hedges g	Welch t-test (p=0.147); normality not checked in any of 5 runs	StatGuard
5	Paired, non-normal differences (lay prompt)	Shapiro on differences (violated) → Wilcoxon primary (p<0.0001), every run	Paired t or Wilcoxon depending on run; primary p-value varied ≈500× (Table 3)	StatGuard (reproducibility)
6	Clean difference, reporting (asked for magnitude)	Welch t (p=0.023), Hedges g=−0.485 with 95% CI, magnitude label	Welch t (p=0.023), Cohen's $d\approx0.49$, mean-difference 95% CI, magnitude label	Tie

group, found it violated for one arm (Shapiro–Wilk $p = 0.002$), and reported the comparison with rank-based context and an effect size. The point is not that the t-test result is necessarily wrong, but that a relevant assumption went unexamined whenever the user did not think to ask—which is precisely the situation of a non-expert user.

The reproducibility gap (Scenario 5). The paired scenario exposes the failure mode most directly. We submitted the identical prompt and dataset in five independent sessions (Table 3; Figure 1). In four runs the analyzer used a paired t-test and reported $p = 0.009$; in one run it first tested the normality of the differences, found it violated, and used a Wilcoxon signed-rank test, reporting $p = 0.000018$—a primary p-value differing by a factor of approximately 500. The divergence is governed entirely by whether that run happened to test an assumption. StatGuard tested the normality of the differences, found it violated, and returned a Wilcoxon signed-rank test as the primary result on every run; it additionally reported the paired-t result as a secondary comparison, making transparent what the alternative method would have yielded.

These two scenarios are illustrative case studies on single datasets, not a controlled large-sample study of error rates; the broad prevalence of such irreproducibility is documented elsewhere [1, 5]. Their role here is to make concrete the property that StatGuard guarantees by construction: the same input yields the same, assumption-appropriate analysis every time.

Table 3: Scenario 5—identical prompt and dataset, five independent runs of the commercial analyzer, versus StatGuard.

Run	Assumption checked?	Test used	Primary p-value
1	No	Paired t-test	0.009
2	Yes (Shapiro–Wilk)	Wilcoxon signed-rank	0.000018
3	No	Paired t-test	0.009
4	No	Paired t-test	0.009
5	No	Paired t-test	0.009
StatGuard	**deterministic**	**Wilcoxon signed-rank**	$\mathbf{< 0.0001}$

5 Discussion

5.1 What the separation does and does not guarantee

StatGuard's guarantees are precise, and it is worth stating their boundary plainly. The framework guarantees that, *given a tool selection and its arguments*, the computation is deterministic, assumption-appropriate by explicit rule, numerically verified against reference implementations, and faithfully reported. It does not remove the language model from the loop entirely: the model still chooses *which* tool to run and *with what arguments*. A poor routing decision—selecting an inappropriate tool, or supplying the wrong column—remains possible.

Two aspects of the design mitigate this residual exposure. First, many statistical decisions that a naive LLM-as-statistician would make by free generation are moved *inside* the deterministic engine: the choice between parametric and non-parametric tests, between equal- and unequal-variance procedures, and between post-hoc methods is made by inspectable rules, not by the model. The model's discretion is correspondingly narrowed. Second, the verification step rejects ill-formed actions—a tool referencing a non-existent column, for instance—and returns a structured error rather than a fabricated result, so that a routing error surfaces as a visible failure rather than a silent miscomputation. The remaining surface—selecting among genuinely applicable analyses—is exactly where an LLM's language understanding is most valuable and least dangerous, because the candidate analyses have all been independently validated.

The distinction is therefore not "LLM versus no LLM," but *where the LLM's stochasticity is permitted to act*. In the LLM-as-statistician paradigm it acts on method choice, assumption checking, computation, and reporting simultaneously; in StatGuard it is confined to high-level routing, with every downstream step deterministic and verified.

5.2 Scope

StatGuard targets standard univariate statistical inference and ordinary least squares regression: the families summarized in Section 4.1. This range covers a large fraction of everyday applied analysis, but it is deliberately bounded. The framework does not attempt logistic or other generalized linear models, factorial designs with interaction effects, time-series, or survival analysis; these are natural extensions that fit the same plugin-and-validation pattern and are left to future work. The design philosophy is that a reproducible analysis system should offer standard, validated methods rather than improvise an arbitrary analysis per request—and consequently its coverage grows by deliberate, validated addition rather than by open-ended generation.

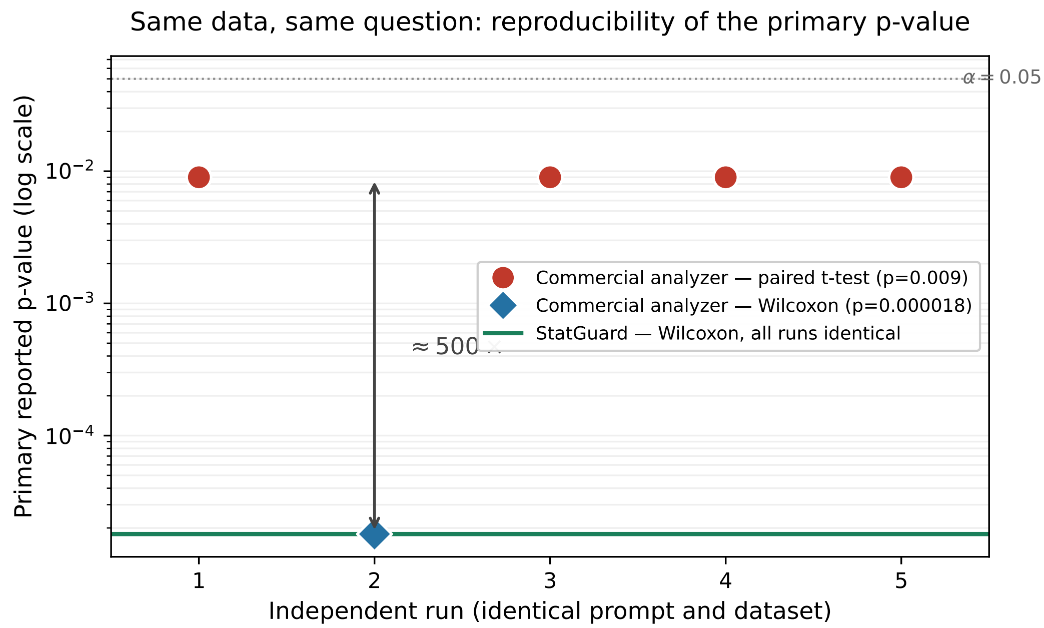

Figure 1: Primary reported p-value across five independent runs of the commercial analyzer on the identical paired-data prompt, versus StatGuard's deterministic result (log scale). Four runs used a paired t-test ($p = 0.009$); one run checked normality and used a Wilcoxon signed-rank test ($p = 0.000018$), a roughly 500-fold difference. StatGuard returns the Wilcoxon result on every run; the single Wilcoxon run of the commercial analyzer coincides with it.

5.3 On the nature of the evidence

The case studies of Section 4.2 are illustrative comparisons on single datasets, chosen to make concrete the failure modes the design addresses; they are not a controlled, large-sample estimate of how often an LLM analyzer errs. The systematic prevalence of analytical irreproducibility in LLMs has been established at scale elsewhere [1, 5], and we rely on that literature for the general claim. Our contribution is the architecture and its validation: the 246-case numerical benchmark is the quantitative evidence for *correctness*, and the determinism of the engine is, by construction, the evidence for *reproducibility*. A larger, pre-registered comparison across many datasets and prompts would strengthen the empirical picture and is a worthwhile direction.

5.4 Implications for high-stakes analysis

The motivation for this work is sharpest in settings where an analysis must be defended, audited, or reproduced by a third party. In scientific research, the credibility of a reported finding rests on the analysis being reproducible: a result that changes when the same data are re-analyzed is difficult to trust and, increasingly, difficult to publish. In clinical and other regulated domains, the analytic pathway from data to conclusion is itself subject to scrutiny, and an analysis that may silently change its method between runs sits poorly with that requirement. The broader concern—that systems based on language models be reliable and auditable when used for consequential decisions—is a recurring theme in current discussion of trustworthy machine learning.

StatGuard is a domain-specific response to that concern. It does not make a language model reliable; it removes the parts of the task that a language model performs unreliably—the computation and the assumption-dependent method choice—and places them in a deterministic, verifiable engine, leaving the model to do what it does well. The principle is general even though the implementation is specific to statistical inference: where correctness is verifiable and reproducibility is required, the verifiable, reproducible component should not be delegated to a sampling-based model. We expect the same separation to be applicable to other analytical domains in which a correct procedure can be specified and checked.

6 Conclusion

We have presented StatGuard, a framework for statistical analysis that retains the natural-language convenience of a language-model interface while guaranteeing that the statistics themselves are deterministic, assumption-appropriate, numerically verified, and faithfully reported. The design rests on a separation of responsibilities: the language model orchestrates—interpreting the request and selecting an analysis—and a deterministic engine, cross-validated against established reference implementations, computes. An anti-fabrication claims ledger ensures that reported numbers are those the engine produced, and a provenance mechanism ties every result to an explicit data version, so that an analysis can be re-executed to an identical result.

We validated the engine against a 246-case benchmark with known ground truth, finding complete agreement, and we used six case studies to illustrate the reproducibility and rigor that motivate the design—including a paired-data scenario in which a commercial LLM analyzer's primary p-value varied roughly five-hundred-fold across identical runs, while StatGuard returned the same assumption-appropriate result every time. The framework is open-source, and its results are reproducible from the public repository.

The approach has clear boundaries: the language model still selects among analyses, the method coverage is deliberately standard, and the comparative evidence is illustrative rather than large-sample. Within those boundaries, StatGuard demonstrates that the reproducibility and verifiability expected of statistical software can be preserved in a system that also accepts open-ended, natural-language requests—and, more generally, that in tasks where correctness can be checked, the checkable part need not, and should not, be left to a stochastic model.

Data and Code Availability

StatGuard is open-source and available at `https://github.com/Cheng-Peng0718/StatGuard_Agent`. The 246-case validation benchmark is regenerated from a fixed random seed via the scripts in the repository, and the six case-study datasets, prompts, and analysis scripts are likewise provided. Session records for the commercial LLM analyzer used in the case studies are archived as supplementary material.

References

[1] Jiaxin Cui and Rohan Alexander. Same prompt, different outcomes: Evaluating the reproducibility of data analysis by LLMs, 2026.

[2] Skipper Seabold and Josef Perktold. statsmodels: Econometric and statistical modeling with Python. In *Proceedings of the 9th Python in Science Conference*, pages 92–96, 2010.

[3] Suriya Sureshkumar. R-LAM: Reproducibility-constrained large action models for scientific workflow automation, 2026.

[4] Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, et al. SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, 17(3):261–272, 2020.

[5] Qiuhai Zeng, Claire Jin, Xinyue Wang, Yuhan Zheng, and Qunhua Li. AIRepr: An analyst-inspector framework for evaluating reproducibility of LLMs in data science, 2025. Findings of EMNLP 2025.